↖ Topic Index

Buffers

Operation	Keystroke	Function	Note
Emacs Buffers ○ List Buffers ○ Switch to Buffer ○ Navigate through buffers in current window • next/previous • with iflipb • same major mode • Buffer Selection ○ Manage Buffers • read-only, clone • rename, kill ○ Accumulate Text • Diff buffer with its file • Locking Buffer • Indirect Buffers • GNU Screen Log Render ○ Buffer View Mode • Edit Binary File: hexl ○ nhexl • strace-mode	Emacs information and edited files are all held inside Emacs buffers. A buffer might be shown inside one (or several) Σ Emacs windows. This table lists the commands you can use to list, manage buffers and also switch the buffer currently shown inside an Emacs window. PEL provides quick access to the relevant customization buffers, several extra buffer-related commands, and quick access to install and activate buffer-related external packages. The **pel-pkg-for-buffer** customization group to control some aspect of Emacs buffers. The user options are: **pel-use-uniquify** : activates **uniquify** to that buffer names show the distinguishing directory after the file name, like this: **`fname|dir`** **pel-use-ascii-table** : activates **ascii-table** external package. See Σ **Help/Info** for the key binding. **pel-use-fzf** : activates **fzf.el** external package. For fast fuzzy search on buffer name. See **fzf manual**, **fzf search syntax**. **pel-use-iflipb** : activates **iflipb** external package. Also activates the **hydra** external package PEL sets **pel-use-hydra** option to **t**. **pel-use-nhexl-mode** : activates **nhexl-mode** external package used to display and manipulate the content of the current buffer in hexadecimal. **pel-use-popup-switcher** : activates **popup-switcher** external package used for popping up a list of buffers. **pel-use-strace** : activates **strace-mode** external package used to display Linux **strace** output. PEL also provides a Hydra that manipulates Emacs windows and buffers. See the Σ **Windows** table for its description. ☝ See Σ **ibuffer-mode** It provides a very powerful buffer listing and manipulation mode, allowing you to list all buffers in a window, filtering them, sorting them, grouping them and operating on them.		
Last updated on:	2026-09-28	See also **Emacs Lisp - Buffers**	
Open this PDF file. See also: Σ **Help/Info**	`<f11> b <f1>`	(**pel-help-pdf** &optional OPEN-WEB-PAGE)	Open the Σ **Buffers** local PDF. With **`C-u`** , **`M--`**, **`M- -1`** or **`C- -1`** prefix: open the remote GitHub hosted PDF instead. If the **pel-flip-help-pdf-arg** user-option is set it's the other way around.
Σ **Customize PEL Buffer Support**	`<f11> b <f2>`	(**pel-customize-pel** &optional OTHER-WINDOW)	Customize PEL Buffer support: open PEL buffer support specific group. • If OTHER-WINDOW is non-nil (use **`C-u`**), display in other window.
Σ **Customize Emacs & external package buffer support**	`<f11> b <f3>`	(**pel-customize-library** &optional OTHER-WINDOW)	Customize Emacs and external packages related to buffer via their customization groups: **Buffer-menu**, **bs**, **ibuffer**, **iflipb**, **minibuffer**, **hexl**, **nhexl**, **popup-switcher**. • When a prefix argument (like **`C-u`**) opens the buffer inside another window.
Show buffer stats, current window last visited buffer	• `<f11> b ?` • `<f11> ? e b`	(**pel-emacs-buffer-stats**)	Show buffer statistics: total count of buffers, # of buffer visiting files, # of special buffers and # of internal buffers. Also show the name of previous buffer used in the current window.
List Buffers	The first 2 commands open a *menu overlaid on the current buffer* that you can use to switch to another buffer: • **buffer-menu-open** is a drop-down hierarchical menu. **psw-switch-buffer** is a pop-up menu. The other commands, list buffers inside a *dedicated buffer/window* and provides a set of commands.		
Open buffer drop-down menu See also: Σ **Menus**	• **`C-<f10>`** • `<f11> b m`	(**buffer-menu-open**)	List buffers in a drop-down menu: lists the buffers by major-mode when several buffers of the same major-mode are opened. Close with **`C-g`** • With Emacs is in ***graphics*** mode this can also be invoked using `C-<down-mouse-1>`
List open buffers in popup menu	`<f11> b b`	(**psw-switch-buffer** &optional ARG)	Show buffers list menu to switch buffer in a popup window menu. • If ARG show only buffers with files and without * in the beginning/end of buffer name. Requires **popup-switcher** PEL activates when **pel-use-popup-switcher** is **t**.
List all buffer used by current Σ Project (Emacs ≥ 29.1)	**`C-x p C-b`**	(**project-list-buffers** &optional ARG)	Display a list of project buffers in a buffer named "*Buffer List*". • By default, all project buffers are listed except those whose names start with a space (for internal use). • With prefix argument ARG, show only buffers that are visiting files.
Show all buffers, including internal hidden buffers ⚠️	`<f11> b a`	(**pel-show-all-buffers** &optional FILES-ONLY)	Display all buffers, including hidden internal buffers, listed inside a *Buffer List* buffer. • With optional argument (as with **`C-u`**) displays only buffer associated with files. Do not manipulate internal buffers unless you understand the consequences. Showing them is meant as an aid for Emacs Lisp code development.
List all buffers using Buffer Menu Mode	• `<f11> b B` • ~~`C-x C-b`~~	• (**buffer-menu** &optional ARG) • (**list-buffers** &optional ARG)	Open the Buffer Menu: lists all buffers except those whose names start with a space (for internal use). • With prefix argument ARG, show only buffers that are visiting files. • buffer-menu also list all single key commands when opening.
The Buffer Menu Columns ➡ Customize via **Buffer-menu**	• C: "." for the buffer from which you came, ">" for buffers you mark to be displayed, and "D" for those you mark for deletion. • R: has a "%" if the buffer is read-only. • M: has a "*" if it is modified, or "S" if you have marked it for saving. • The remaining columns show the buffer name, the buffer size in characters, its major mode, and the visited file name (if any).		
Buffer Menu Mode keys	• **`?`** : Get help : Immediately • **`g`** : Update buffer list : immediately • **`C-n`** : next buffer in list : immediately • **`SPC`** : next buffer in list : immediately • **`n`** : next buffer in list : immediately • **`C-p`** : previous buffer in list : immediately • **`p`** : previous buffer in list : immediately • **`C-d`** : mark buffer for deletion : deleted when pressing **`x`** • **`d`** : mark buffer for deletion : deleted when pressing **`x`** • **`k`** : mark buffer for deletion : deleted when pressing **`x`** • **`s`** : save buffer : saved when pressing **`x`** • **`<DEL>`** : Move to previous line, remove all marks on buffer : immediately if just after marking • **`M-<DEL>`** : Remove a specific mark from all buffers : immediately if just after marking • **`u`** : unmark all marks on buffer : immediately • **`x`** : execute marked commands (delete buffers marked for deletion) : immediately • **`~`** : mark buffer as un-modifiable : immediately • **`%`** : toggle read-only : immediately • **`1`** : display emacs in full emacs screen : immediately • **`2`** : Display this buffer & next in horizontal window : immediately • **`o`** : replace other (next) window with this buffer : immediately • **`m`** : mark buffer to be displayed in windows : when pressing v • **`v`** : display buffers marked with in as many windows as required : immediately • **`q`** : quit buffer list : immediately		
List buffers using iBuffer Mode ★★ See Σ **ibuffer-mode** for more info on ibuffre-mode.	**`C-x C-b`**	(**ibuffer** &optional OTHER-WINDOW-P NAME QUALIFIERS NOSELECT SHRINK FILTER-GROUPS FORMATS)	Begin using Ibuffer to show, edit and operate on a list of buffers. • PEL binds '**ibuffer**' to **`C-x C-b`** key sequence . • ibuffer provides more functionality, working like dired, allowing to sort by name, size, mode, filtering by mode (hit return on the mode of a buffer). Type **`<f1> m`** to get the list of possible actions that can be done on the listed buffers.

Operation	Keystroke	Function	Note
Switch to Buffer	The following commands switch to another buffer inside the current window. The function invoked by the key sequence depends on the current input completion. All commands prompt for a buffer and use the currently active input completion system. • See Σ **Completion/Input**. To show completion used, type: `<f11> M-c ?` To change the main completion mode, type: `<f11> M-c <f4>`		
Switch to buffer Change buffer in current window.	`C-x b`	(**switch-to-buffer** BUFFER-OR-NAME &optional NORECORD FORCE-SAME-WINDOW)	Switch window to display the previous, or another buffer (entered at echo area prompt). ☝ The invisible buffers have a name that start with a space. To see them type space and tab and a list of those buffers will appear before the list of visible buffers.
• With ido		(**ido-switch-buffer**)	Switch to another buffer. The buffer is displayed according to '**ido-default-buffer-method**' -- the default is to show it in the same window, unless it is already visible in another frame.
		As you type in a string, all of the buffers matching the string are displayed if substring-matching is used (default). Look at '**ido-enable-prefix**' and '**ido-toggle-prefix**'. When you have found the buffer you want, it can then be selected. As you type, most keys have their normal keybindings, except for the following: `RET` Select the buffer at the front of the list of matches. If the list is empty, possibly prompt to create new buffer. `C-j` Use the current input string verbatim. `C-s` Put the first element at the end of the list. `C-r` Put the last element at the start of the list. `TAB` Complete a common suffix to the current string that matches all buffers. If there is only one match, select that buffer. If there is no common suffix, show a list of all matching buffers in a separate window. `C-e` Edit input string. `C-x C-b` Fallback to non-ido version of current command. `C-t` Toggle regexp searching. `C-p` Toggle between substring and prefix matching. `C-c` Toggle case-sensitive searching of buffer names. `?` Show list of matching buffers in separate window. `C-x C-f` Drop into 'ido-find-file'. `C-k` Kill buffer at head of buffer list. `C-a` Toggle ignoring buffers listed in 'ido-ignore-buffers'.	
• With ivy		(**ivy-switch-buffer**)	Switch to another buffer. Uses Ivy to show buffer names.
• With helm		(**helm-mini**)	Uses helm to list current buffer but also list identified by '**helm-mini-default-sources**' which defaults to (helm-source-buffers-list helm-source-recentf helm-source-buffer-not-found).
Open current Σ Project's buffer in current window.	`C-x p b`	(**project-switch-to-buffer** BUFFER-OR-NAME)	Display buffer BUFFER-OR-NAME in the currently selected window. • When called interactively, prompts for a buffer belonging to the current project. Two buffers belong to the same project if their project instances, as reported by 'project-current' in each buffer, are identical.
Open matching buffer of another Σ Project (Emacs≥31.1 | project≥0.12)	`C-x p M-d`	(**project-find-matching-buffer**)	Switch to a matching buffer in another project. • See also **pel-open-file-in-other-dir** bound to `<f11> f M-d` in Σ **File-mngt**
	• For most file-visiting buffers, the matching buffer is one visiting a file in the other project which has the same file name relative to the project root. • See `**project-find-matching-file**' for details. • Non-file-visiting major modes may configure a different notion of matching buffer; see `**project-find-matching-buffer-function**'. • When called during switching to another project, this command will detect that, and use the override. Otherwise, it prompts for the project to use from the list of known projects. • When calling from Lisp, bind `**project-current-directory-override**' to a directory under the target project to preempt this prompting.		
Switch to an M Org-Mode buffer in current window.	• `<f11> b o` • `<f11> M-o b`	(**org-switchb** &optional ARG)	Switch between Org buffers. Available when **pel-use-org** is set. • With '`C-u`' prefix: restrict available buffers to files. • With '`C-u C-u`' prefix: restrict available buffers to agenda files.
Switch buffer with fzf See: Σ **File-mngt**, **fzf**, **fzf search syntax**	• `<f11> b z` ✱ `<f7> b M-z`	(**fzf-switch-buffer**)	Select buffer for current window with a fzf-driven fuzzy search. • Uses the **fzf command line utility** for fast & flexible search. 📦 Requires the **fzf.el** external package ☑ activated by **pel-use-fzf** .
this-window-prefix	`C-x 4 1`	This general prefix command requests the buffer of the next command to be displayed in the same window.	
Open Buffer in another window	The following commands display an opened buffer into another window. All commands prompt for a buffer and use the currently active input completion system. • See Σ **Completion/Input**. To show completion used, type: `<f11> M-c ?` To change the main completion mode, type: `<f11> M-c <f4>`		
Switch buffer in other window (and with helm)	• `C-x 4 b` • `<f11> w B`	(**switch-to-buffer-other-window** BUFFER-OR-NAME &optional NORECORD)	Select buffer bufname in another window (switch-to-buffer-other-window). See Select Buffer.
• With ido		(**ido-switch-buffer-other-window)**	Switch to another buffer and show it in another window. The buffer name is selected interactively by typing a substring. For details of keybindings, see '**ido-switch-buffer**'.
• With ivy		(**ivy-switch-buffer-other-window**)	Switch to another buffer in another window.
Display buffer in other window, without selecting it.	• `C-x 4 C-o` • `<f11> w b`	(**display-buffer** BUFFER-OR-NAME &optional ACTION FRAME)	Display a buffer in other window but don't select it. • Used with Emacs default completion, ivy and helm.
• With ido		(**ido-display-buffer** &optional ACTION)	Display a buffer in another window but don't select it. • With `C-u` prefix: display the buffer in another window even if it's already displayed in the current window.
Display Σ Project buffer in another window, without selecting it.	`C-x 4 p C-o`	(**project-display-buffer** BUFFER-OR-NAME)	Display BUFFER-OR-NAME in some window, without selecting it. • When called interactively, prompts for a buffer belonging to the current project. Two buffers belong to the same project if their project instances, as reported by 'project-current' in each buffer, are identical. • Uses '**display-buffer**' as a subroutine, which see for how it is determined where the buffer will be displayed.
other-window-prefix	`C-x 4 4`	A more general prefix command affects the buffer displayed by a subsequent command invoked after this prefix command (other-window-prefix). It requests the buffer displayed by a subsequent command to be shown in another window	
Open Buffer into	**... another tab.** See Σ **Tab Bar**		
Open buffer in new tab • See Σ **Tab Bar**	• `C-x t b` • `<M-f11> M-= M-b`	(**switch-to-buffer-other-tab** BUFFER-OR-NAME)	Prompt for a buffer and switch to buffer BUFFER-OR-NAME in another tab. • Like `C-x 5 b` , but creates a new tab.
Open Buffer into	**... another frame**		
Switch to buffer in other frame (Emacs, ivy, helm)	• `C-x 5 b` • `<f11> F O`	(**switch-to-buffer-other-frame** BUFFER-OR-NAME &optional NORECORD)	Switch to buffer BUFFER-OR-NAME in another frame. Uses Emacs, ivy or helm input completion. BUFFER-OR-NAME may be a buffer, a string (a buffer name), or nil. Return the buffer switched to.
• With ido		(**ido-switch-buffer-other-frame**)	Switch to another buffer and show it in another frame. The buffer name is selected interactively by typing a substring. For details of keybindings, see '**ido-switch-buffer**'.
Display Σ Project buffer in another frame.	`C-x 5 p C-o`	(**project-display-buffer-other-frame** BUFFER-OR-NAME)	Display BUFFER-OR-NAME preferably in another frame. • When called interactively, prompts for a buffer belonging to the current project. Two buffers belong to the same project if their project instances, as reported by 'project-current' in each buffer, are identical. • Uses '**display-buffer-other-frame**' to display selected buffer into another frame.

Operation	Keystroke	Function	Note
Navigate through Buffers in current window	PEL provides the pel-∑buffer **Hydra** the gives quick access to commands that changes the buffer shown in the current window. • To gain access to the keys, type `<f7> b` key sequence to start the pel-∑buffer Hydra. • Then type the keys listed in the Hydra table below. Stop the Hydra with `<f7>` . It can also be stopped by using the fzf command key: `M-z` 📦Requires the **hydra** external package PEL provides Hydra when **pel-use-hydra** or the **pel-use-iflipb** user option is set to **t**. ☝ For all buffer selection commands you can also use the repeat command key, `C-x z` or `<f5>` to repeat the last command quickly. See: **Undo/Redo**		

Activate the pel-∑buffer Hydra
★★
pel-use-fzf must be turned on for the buffer selection done by fzf.
See **Windows**
	✱ `<f7> b`		`Buffer management hydra; foreign keys allowed:`
			`Buffer | Buffer Selection | Flip Recently Viewed | Same Mode | Other`
			`---------- | ---------- | ---------- | ---------- | ----------`
			`M-n: next | M-s: window | C-n: next | ]: next | ?: hint`
			`M-p: prev | M-.: next | C-p: prev | [: previous | <f7>: cancel`
			`M-l: last | M-,: prev |  |  |`
			`M-v: view | M-z: fzf & close |  |  |`
			Other keys can be typed when the **pel-∑buffer** Hydra is active. The Hydra menu shows at the bottom of the frame. Type `?` to toggle displaying it.
			☝You may want to switch to the **pel-∑window Hydra** to change and manage window(s) by typing `<f7> <f7> w`

• **Next/Previous Buffer**	The following commands change current buffer to next or previous buffer, or to what was used last. The commands are accessible through the **pel-∑buffer** Hydra when the **hydra** package is used. They are also always available from global key sequences.		
Switch to next buffer	• `C-x <right>` • `C-x C-<right>` • `<f11> b n` ✱ `<f7> b M-n`	(**next-buffer**)	Switch to the next buffer displayed in the current window.
Switch to previous buffer	• `C-x <left>` • `C-x C-<left>` • `<f11> b p` ✱ `<f7> b M-p`	(**previous-buffer**)	Switch to the previous buffer displayed in the current window. • This command is also available in the **pel-∑buffer Hydra** as `M-p`
Switch to previous buffer in window	`<f11> b l` ✱ `<f7> b M-l`	(**pel-switch-to-last-used—buffer**)	Switch buffer in current window to the buffer previously seen in this window. Used twice returns to the same buffer.
• **iflipb: Flip to next/ previous recently visited buffer**	The following commands let you flip between recently visited buffers in a way that resembles what Alt-Tab and Alt-Shift-Tab does on Windows. • A list of buffers is shown in the minibuffer at the bottom of the screen when you use the command. You can see them in the **pel-∑buffer** Hydra above. • You can also identify buffer filtering in the iflipb customization group (use `<f11> b <f3>` and select iflipb to access it). 📦 This requires the **iflipb** external package PEL activates it when **pel-use-iflipb** user-option is turned on (set to **t**). This also forces activation of the **hydra** package because the **iflipb** commands are bound to the **pel-∑buffe**r Hydra, allowing quick single keystroke access without the use of a prefix key.		
Flip to next buffer	• `<f11> b C-n` ✱ `<f7> b C-n`	(**iflipb-next-buffer** ARG)	Flip to the next buffer in the buffer list. • Consecutive invocations switch to less recent buffers in the buffer list. • Buffers matching 'iflipb-always-ignore-buffers' are always ignored. • Without a prefix argument, buffers matching 'iflipb-ignore-buffers' are also ignored.
Flip to previous buffer	• `<f11> b C-p` ✱ `<f7> b C-p`	(**iflipb-previous-buffer**)	Flip to the previous buffer in the buffer list. Consecutive invocations switch to more recent buffers in the buffer list.
• **Cycle Trough buffers using same major-mode**	The following 2 PEL commands cycle the buffers using the same major mode inside the current window. • This works with any mode, including special buffer modes. ☝ The key bindings that start with `<f7> b` starts a **pel-∑buffer Hydra**. After the initial key sequence, single key commands are available.		
Show next buffer using same major mode	• `<f11> b ]` ✱ `<f7> b ]`	(**pel-smb-next** &optional REFRESH)	Open next buffer of same major-mode from the registered list. • Refresh list when wrapping. With optional prefix argument: REFRESH the list of buffers.
Show previous buffer using same major mode	• `<f11> b [` ✱ `<f7> b [`	(**pel-smb-previous** &optional REFRESH)	Open previous buffer of same major-mode from the registered list. • Refresh list when wrapping. With optional prefix argument: REFRESH the list of buffers.
• **Buffer Selection**	The Buffer Selection command opens a small window listing currently opened buffers. You can select a buffer from it but also manage buffers. • These commands are using the built-in **bs.el** library. PEL provides extra commands that extends it. ☝The Buffer Selection commands key bindings are also available through the **pel-∑buffer** Hydra when the **hydra** package is used.		
Show next buffer in selection	• `<f11> b .` ✱ `<f7> b M-.`	(**pel-bs-next**)	Show next buffer in current window. • Next buffer is selected by the list of buffers selected by the Buffer Selection Mode configuration and sorting order last identified. These can be controlled by opening the Buffer Selection Mode with the **bs-show** command and then using the commands available in this mode, such as: **bs-select-next-configuration**, **bs-toggle-show-all** and **bs-show-sorted**.
Show previous buffer in selection	• `<f11> b ,` ✱ `<f7> b M-,`	(**pel-bs-previous**)	Show previous buffer in current window. • Next buffer is selected by the same criteria as for **pel-bs-next** described above.
Customize buffer selection mode	• `<f11> b S`	(**bs-customize**)	Customization of group bs for Buffer Selection Menu. • Active configuration can be changed in the bs-show buffer. See below.
Show Buffer Selection	• `<f11> b s`	(**bs-show** ARG)	Open the **bs-mode** buffer by splitting the current window • Shows menu of buffers to select and manipulate buffers. • With no prefix argument: show buffers selected by the **default** configuration. • If a `C-u` prefix argument: show buffers selected by **alternative** configuration. • With numeric argument (`M-1`, `M-2`,…) show buffers selected by the **nth** configuration. • The pre-configured configurations are: all, files, file-and-scratch, all-intern-last.

• **Buffer Selection Menu (bs-mode) commands**	• There are many key commands similar to 'Buffer-menu-mode' for manipulating the buffer list and the buffers themselves. User can move with [up] or [down], select a buffer by RET or [SPC]
	  • Type q to leave Buffer Selection Menu without a selection.
	  • Type ? after invocation to get help on commands available.
	  • With prefix argument ARG show a different buffer list. Function 'bs--configuration-name-for-prefix-arg' determine accordingly name of buffer configuration.		

Select current line's buffer : visit buffer	• `RET` • `SPC` • `f`	(**bs-select**)	Select current line's buffer and other marked buffers. • If there are no marked buffers the window configuration before starting Buffer Selection Menu will be restored. If there are marked buffers each marked buffer and the current line's buffer will be selected in a window. • Leave Buffer Selection Menu.
View current line's buffer	`v`	(**bs-view**)	View current line's buffer in View mode. A minor mode for viewing text but not editing it. • See Buffer View Mode below in this table. • Leave Buffer Selection Menu.
Move point down	• `n` • `<down>`	(**bs-down** ARG)	Move point vertically down ARG lines in Buffer Selection Menu.
Move point up	• `p` • `<up>`	(**bs-up** ARG)	Move point vertically up ARG lines in Buffer Selection Menu.
Refresh Buffer Selection Menu	`g`	(**bs-refresh** &rest IGNORED)	Refresh whole Buffer Selection Menu.
Bury buffer	`b`	(**bs-bury-buffer**)	Bury (not kill!) the buffer on this line: move it to the end of buffer list.
Cycle through buffer line sorting method	`S`	(**bs-show-sorted**)	Show buffer list sorted by next sort aspect. • The sort methods supported are: by name, by size, by mode, by filename, by nothing (not sorted).
Toggle show all buffers	`a`	(**bs-toggle-show-all**)	Toggle show all buffers / show buffers with current configuration.

Operation	Keystroke	Function	Note
Cycle through buffer selection configuration : types of buffers iterated through	`c`	(**bs-select-next-configuration** &optional START-NAME)	Apply next configuration START-NAME and refresh buffer list. • If START-NAME is nil the current configuration 'bs-current-configuration' will be used. • Cycles through the following methods of buffer cycling: all, files, files-and-scratch, all-intern-last
Prompt for buffer selection configuration	`C`	(**bs-set-configuration-and-refresh**)	Ask user for a configuration and apply selected configuration. • Supports tab-based completion. • Refresh whole Buffer Selection Menu.
Add a new configuration to display only buffers of this major mode	`.`	(**pel-bs-this-mode-only**)	Add a Buffer Selection configuration for buffer of this mode only. • Add a Buffer Selection that will be named "only-X" where X is the major mode of the current line buffer. • This configuration will only show buffers that use the same major mode.
	☝To iterate through buffers of a specific major mode only, do this: • Open the Buffer Selection Mode buffer: execute bs-show: type `<f11> b s` • Move point to a buffer line of the wanted major-mode. • If that mode is currently not displayed, change the configuration to **all** by typing `C all RET` first and then move point to the proper line. • Hit `.` to execute **pel-bs-thi-mode-only** and select the major mode of interest. • Hit `RET` to display the buffer and dismiss the Buffer Selection Mode buffer. • To select the next or previous buffer of the same major mode activate the **pel-∑buffer** Hydra by typing `<f7> b M-.` or `<f7> b M-,` continue with `M-.` or `M-,`		
Open selected buffer in other window	`o`	(**bs-select-other-window**)	The window configuration before starting Buffer Selection Menu will be restored unless there is no other window. In this case a new window will be created. • Leave Buffer Selection Menu.
Open selected buffer in other window - stay in Buffer Selection buffer	`C-o`	(**bs-tmp-select-other-window**)	Make the other window select this line's buffer. • The current window remains selected.
Save buffer	`s`	(**bs-save**)	Save buffer on current line.
Kill buffer	`k`	(**bs-delete**)	Kill buffer on current line.
Toggle buffer read-only status	`%`	(**bs-toggle-readonly**)	Toggle read-only status for buffer on current line. • Uses function 'read-only-mode'.
Clear buffer modified-flag	`~`	(**bs-clear-modified**)	Set modified flag for buffer on current line to nil. ⚠️ Be sure you don't want to save these modifications: Emacs won't prompt you for that modified buffer when closing.
Visit tags table file	`t`	(**bs-visit-tags-table**)	Visit the tags table in the buffer on this line. See 'visit-tags-table' bound to `<f11> X t` in PEL. See ∑ **Xref** - Cross References.
Mark line's buffer to be displayed	`m`	(**bs-mark-current** COUNT)	Mark buffers. Move point vertically down COUNT lines. • COUNT is the number of buffers to mark.

Mark line's buffer to show always	`+`	(**bs-set-current-buffer-to-show-always** &optional NOT-TO-SHOW-P)	Toggle status of buffer on line to 'always shown'.
			• NOT-TO-SHOW-P: prefix argument.
			  • With no prefix argument the buffer on current line is marked to show always.
			  • Otherwise it is marked to show never.

Toggle line's buffer show always/never/ normal	`M`	(**bs-toggle-current-to-show**)	Toggle status of showing flag for buffer in current line through: never show, always show, show normally
Unmark previous line buffer to be displayed	`DEL`	(**bs-unmark-previous** COUNT)	Unmark previous COUNT buffers. • Move point vertically up COUNT lines. • When called interactively a numeric prefix argument sets COUNT.
Unmark line's buffer to be displayed	`u`	(**bs-unmark-current** COUNT)	Unmark buffers. Move point vertically down COUNT lines. • COUNT is the number of buffers to unmark.
Unmark all buffer lines	`U`	(**bs-unmark-all**)	Unmark all buffers.
Scroll right	• `>` • `C-x >`	(**scroll-right** &optional ARG SET-MINIMUM)	Scroll selected window display ARG columns right. • Default for ARG is window width minus 2.
Scroll left	• `<` • `C-x <`	(**scroll-left** &optional ARG SET-MINIMUM)	Scroll selected window display ARG columns left. • Default for ARG is window width minus 2.
Close Buffer-Selection-Menu	• `q` • `C-c C-c`	(**bs-kill**)	Let buffer disappear and reset window configuration.
Abort	• `C-g` • `C-]`	(**bs-abort**)	Ding and leave Buffer Selection Menu without a selection.
Display Help	`?`	(**bs-help**)	Display help in the Help buffer. ⚠️ This conflicts with PEL **pel-∑buffer** hint key which takes precedence.
Manage Buffers	The following commands support buffer management: display information, change read-only mode, clone buffer, rename buffer, kill buffer, etc…		
Toggle read-only status of buffer	• `C-x C-q` • `<f11> b r`	(**read-only-mode** &optional ARG)	When the buffer is in read-only mode the mode line shows '%%' on the left side, in the 'ch' area of *"cs:ch-fr buf pos line (major minor)"*.
	The manual states: *"For a read-only buffer, it shows '%*' if the buffer is modified, and '%%' otherwise."* ☛ **See also:** the **View Mode** activating commands toward the end of this table. A buffer in View Mode cannot be modified. • The View Mode may be used to ensure that no modifications are made to a buffer (visiting a file or not).		
Clone buffer		(**clone-buffer** &optional NEWNAME DISPLAY-FLAG)	Create and return a twin copy of the current buffer.
	Unlike an indirect buffer, the new buffer can be edited independently of the old one (if it is not read-only). NEWNAME is the name of the new buffer. It may be modified by adding or incrementing <N> at the end as necessary to create a unique buffer name. • For example if buffer *Help* is opened it opens another one named *Help*<2> (or *Help*<3> if *Help*<2> already exists, etc…)		
Rename a buffer	`<f11> b R`	(**rename-buffer** NEWNAME &optional UNIQUE)	Rename current buffer. • With optional **C-u** prefix argument, the name is auto generated to be unique.
Rename buffer - use unique name	`<f11> b U`	(**rename-uniquely**)	Rename the current buffer by adding '*<number>*' to the end. • Use this if you want multiple *Buffer* or *Info* buffers for example. • Example: StackExchange: How can I have multiple help buffer with different content
Kill current buffer See also: ∑ **Windows**	• `<f11> b k` • `⌘-k` • `⌘-&`	(**kill-current-buffer**)	Kill (close) the current buffer. Does not prompt if there is no change in the buffer. • PEL also provides a window management Hydra with ability to kill the current buffer. See ∑ **Windows** for more info.
Kill buffer See also: ∑ **Windows**	• `C-x k` ✱ `<f7> k`	(**kill-buffer** &optional BUFFER-OR-NAME)	Kill (close) the current buffer. Prompt to identify a buffer unless inside a Window Hydra which does not prompt.
		(**iflipb-kill-buffer**)	When the **iflipb** external package, activated by **pel-use-iflipb** is active, the key is bound to the iflip-kill-buffer which invokes the kill-buffer and maintains the iflipb buffer list state.
Kill current buffer and close window See also: ∑ **Windows**	• `C-x 4 0` ✱ `<f7> K`	(**kill-buffer-and-window**)	Kill the current buffer and delete the selected window. • PEL also provides a window management Hydra when the **hydra** package is used. with ability to kill the current buffer and close windows in separate operations.
Kill some buffer		(**kill-some-buffers** &optional LIST)	Kill some buffers. Asks the user whether to kill each one of them.
Delete all windows of a specific buffer		(**delete-windows-on** &optional BUFFER-OR-NAME FRAME)	Deletes all windows showing BUFFER-OR-NAME, by calling 'delete-window' on those windows.

Operation	Keystroke	Function	Note
Kill all current project buffers (Emacs ≥ 29.1) See Σ **Project**	`C-x p k`	(**project-kill-buffers** &optional NO-CONFIRM)	Kill the buffers belonging to the current project. Two buffers belong to the same project if their project instances, as reported by 'project-current' in each buffer, are identical. Only the buffers that match a condition in 'project-kill-buffer-conditions' will be killed. • If NO-CONFIRM is non-nil, the command will not ask the user for confirmation. • NO-CONFIRM is always nil when the command is invoked interactively.
Save some project buffers (Emacs≥31.1 | project≥0.12)	`C-x p C-s`	(**project-save-some-buffers** ARG)	Like '**save-some-buffers**' (bound to `C-x s`, see Σ **File-mngt**) , but only for this project's buffers. See Σ **Project**
Accumulating Text	Emacs provides the following commands to **insert text** in other buffers, current buffer, or a file.		
Append region to specified buffer	`<f11> b t a`	(**append-to-buffer** BUFFER START END)	Append marked region of current buffer into the BUFFER specified at prompt before it's point. • BUFFER: a buffer or the name of a buffer. Creates BUFFER if it doesn't already exist.
Prepend region to specified buffer	`<f11> b t p`	(**prepend-to-buffer** BUFFER START END)	Prepend marked region of current buffer into the BUFFER specified at prompt after it's point. • BUFFER: a buffer or the name of a buffer. Creates BUFFER if it doesn't already exist.
Copy region to specified buffer (replacing old content)	`<f11> b t c`	(**copy-to-buffer** BUFFER START END)	Copy to specified BUFFER the text of the region. • The text is inserted into that buffer, replacing existing text there. • BUFFER: a buffer or the name of a buffer. Creates BUFFER if it doesn't already exist.
Insert content of specified buffer at point	`<f11> b t i`	(**insert-buffer** BUFFER)	Insert after point the contents of BUFFER. • Puts mark after the inserted text. • BUFFER may be a buffer or a buffer name.
Append region's text to specified file	`<f11> b t f`	(**append-to-file** START END FILENAME)	Append the contents of the region to the end of file FILENAME. • This does character code conversion and applies annotations like 'write-region' does.
Diff buffer with file	Use these commands to see the difference between the buffer and its file when a buffer is rising a file and the changes were not saved back to the file. See also: Σ **Diff & Merge**		
Diff buffer with its file	• `<f11> b =` • `<f11> d b`	(**diff-buffer-with-file** &optional BUFFER	View the differences between BUFFER and its associated file. • This requires the external program 'diff' to be in your '**exec-path**'.
edit buffer with its file	• `<f11> b M-=` • `<f11> d e b f`	(**ediff-current-file**)	Start ediff between current buffer and its file on disk. • This command can be used instead of 'revert-buffer'. • If there is nothing to revert then this command fails.
Locking Buffer	Lock a buffer against: exit, kill or all (both) to prevent accidental killing of the buffer. Controlled by 'emacs-lock-default-locking-mode' normally set to all.		
Toggle emacs lock-mode	`<f11> b L`	(**emacs-lock-mode** &optional ARG)	Toggle Emacs Lock mode in the current buffer. With prefix arg: prompt for locking mode. • Normally set to all: buffer locked against killing buffer and exiting Emacs.

Indirect Buffers
☝ ☛
	As described in Emacs Indirect Buffer section, "an indirect buffer shares the text of some other buffer, called the base buffer of the indirect buffer.
	• In some ways it is a buffer analogue of a **symbolic link** between files.
	  • The text of the indirect buffer is always identical to the text of its base buffer; changes made by editing either one are visible immediately in the other.
	• But in all other respects, the indirect buffer and its base buffer are completely separate.
	  • They can have different names, different values of point, different narrowing, different markers, different major modes, and different local variables."
	
	Use indirect buffers to show the same file in 2 or more windows but want to narrow an area in 1 buffer while seeing the complete text in the other window.		

Create indirect buffer explicitly	`<f11> b I m`	(**make-indirect-buffer** BASE-BUFFER NAME &optional CLONE)	Create and return an indirect buffer for buffer BASE-BUFFER, named NAME. • BASE-BUFFER should be a live buffer, or the name of an existing buffer. • NAME should be a string which is not the name of an existing buffer. • Optional argument CLONE non-nil means preserve BASE-BUFFER's state, such as major and minor modes, in the indirect buffer. • CLONE nil means the indirect buffer's state is reset to default values.
Create indirect buffer of current buffer	`<f11> b I c`	(**clone-indirect-buffer** NEWNAME DISPLAY-FLAG &optional NORECORD)	Create an indirect buffer that is a twin copy of the current buffer.
	• Give the indirect buffer name NEWNAME. Interactively, read NEWNAME from the minibuffer when invoked with a prefix arg. If NEWNAME is nil or if Not sourced with a prefix arg, NEWNAME defaults to the current buffer's name. The name is modified by adding a '<N>' suffix to it or by incrementing the N in an existing suffix. Trying to clone a buffer whose major mode symbol has a non-nil 'no-clone-indirect' property results in an error. • DISPLAY-FLAG non-nil means show the new buffer with 'pop-to-buffer'. This is always done when called interactively. • Optional third arg NORECORD non-nil means do not put this buffer at the front of the list of recently selected ones.		
Create indirect buffer of current buffer in another window	• `C-x 4 c` • `<f11> b I w`	(**clone-indirect-buffer-other-window** NEWNAME DISPLAY-FLAG &optional NORECORD)	Like 'clone-indirect-buffer' but display in another window.
GNU Screen Log File	GNU Screen log files contain ANSI escape codes normally interpreted by the shell but not by Emacs. Use the following command to render the codes.		
Fix the rendering of the log file created by GNU Screen See also: • Σ **Text Modes** • 🚦 **Shells/Terminals Comparisons** for info on GNU Screen	`<f11> t s`	(**pel-screen-log-fix-rendering**)	Fix rendering of buffer created by the GNU Screen log. • It converts the marked area of a buffer, if it is marked, otherwise it processes the entire or the narrowed portion of the buffer. It renders the escape codes, converts the line endings to Unix-style line endings. • In some cases the log file created by GNU Screen injects one extra line per line. If the '**pel-screen-log-delete-all-consecutive-blank-lines**' user-option is set, the function removes them. • Some artifact main remain after these transformations. To fix them automatically, identify a set of regular expression/replacement string pairs in the '**pel-screen-log-fix-regexp-pairs**' user-option. ⚠️ This removes the escape codes from the buffer but renders them using color and other attributes. If you save the file these will no longer be visible unless you first activates the enriched text mode in the buffer to encode the text attributes in a way Emacs will be able to interpret later.

Operation	Keystroke	Function	Note
Buffer View Mode	Several commands (view-buffer, etc.., see at top of this table) activate the View Mode for a buffer where the buffer is essentially read-only and special commands are available. • Emacs commands editing the buffer contents are not available; instead, a special set of commands (mostly letters and punctuation) are defined for moving around in the buffer. • Space scrolls forward, Delete scrolls backward. • Type H for a list of all View commands. See the View Mode command list below.		
Prompt for buffer to view in view-mode	`<f11> b V`	(**view-buffer** BUFFER &optional EXIT-ACTION)	View BUFFER in View mode, returning to previous buffer when done. • Prompt for buffer to open in view-mode.
Toggle Buffer view-mode	`<f11> b v` ✱ `<f7> b M-v`	(**view-mode** &optional ARG)	Toggle view-mode. • When View mode is enabled, commands that do not change the buffer contents are available as usual. Kill commands save text but do not delete it from the buffer. Most other commands beep and tell the user that the buffer is read-only. • The `<f7> b M-v` Hydra key sequence is available when the **hydra** package is used.
View Mode commands • **In view-mode no modifications are allowed.**	**H, h, ?** Show this message. **Digits** provide prefix arguments. **-** negative prefix argument. **<** move to the beginning of buffer. **>** move to the end of buffer. **o** scroll so that buffer end is at last line of window. **SPC** scroll forward "page size" lines. With prefix scroll forward prefix lines. **DEL, S-SPC** scroll backward "page size" lines. With prefix scroll backward prefix lines. **z** like SPC but with prefix sets "page size" to prefix. **w** like DEL but with prefix sets "page size" to prefix. **d** scroll forward "half page size" lines. With prefix, sets "half page size" to prefix lines and scrolls forward that much. **u** scroll backward "half page size" lines. With prefix, sets "half page size" to prefix lines and scrolls backward that much. **RET, LFD** scroll forward one line. With prefix scroll forward prefix line(s). **y** scroll backward one line. With prefix scroll backward prefix line(s). **F** revert-buffer if necessary and scroll forward. Use this to **view a changing file**. **=** prints the current line number. **%** goes prefix argument (default 100) percent into buffer. **g** goes to line given by prefix argument (default first line). **.** set the mark. **x** exchanges point and mark. **@** return to mark and pops mark ring. Mark ring is pushed at start of every successful search and when jump to line occurs. The mark is set on jump to buffer start or end. **m** save current position in character register. **'** go to position saved in character register. **s** do forward incremental search. **r** do reverse incremental search. **/** searches forward for regular expression, starting after current page. ! and @ have a special meaning at the beginning of the regexp: ! means search for a line with no match for regexp. @ means start search at beginning (end for backward search) of buffer. **** searches backward for regular expression, starting before current page. **n** searches forward for last regular expression. **p** searches backward for last regular expression. **q** quit View mode, restoring this window and buffer to previous state. q is the normal way to leave view mode. **e** exit View mode but stay in current buffer. Use this if you started viewing a buffer (file) and find out you want to edit it. This command restores the previous read-only status of the buffer. **E** exit View mode, and make the current buffer editable even if it was not editable before entry to View mode. **Q** quit View mode, restoring all windows to previous state. **c** quit View mode and maybe switch buffers, but don't kill this buffer. **C** quit View mode, kill current buffer and go back to other buffer. The effect of **c**, **q** and **C** depends on how view-mode was entered. • If it was entered by **view-file**, **view-file-other-window**, **view-file-other-frame,** or M-x **dired-view-file** (M-x view-file, M-x view-file-other-window, M-x view-file-other-frame, or the Dired mode v command), then **q** will try to kill the current buffer. • If view-mode was entered from another buffer, by `<f11> b v`, M-x view-buffer-other-window, M-x view-buffer-other frame, M-x view-file, M-x view-file-other-window, or M-x view-file-other-frame, then **c**, **q** and **C** will return to that buffer.		
Edit Binary file with hexl	Emacs provides the built-in **hexl** mode to edit files in hexadecimal mode. To use it you must: • use the hexl-find-file to open the file in binary mode, or • use the hexl-mode command to convert an already opened buffer. To **exit** this mode and go back to the original mode type `C-c C-c`		
Open a file in hexl-mode See also: **File-mngt**	`<f11> f M-x`	(**hexl-find-file** FILENAME)	Edit file FILENAME as a binary file in hex dump format. • Switch to a buffer visiting file FILENAME, creating one if none exists, and edit the file in 'hexl-mode'.
Toggle hexl mode	`<f11> b M-x`	(**hexl-mode** &optional ARG)	Toggle the hexl mode: a mode for editing binary files in hex dump format.
	• This is not an ordinary major mode; it alters some aspects of the current mode's behavior, but not all; also, you can exit Hexl mode and return to the previous mode using 'hexl-mode-exit'. • This function automatically converts a buffer into the hexl format using the function 'hexlify-buffer'. • Each line in the buffer has an "address" (displayed in hexadecimal) representing the offset into the file that the characters on this line are at and 16 characters from the file (displayed as hexadecimal values grouped every 'hexl-bits' bits, and as their ASCII values). • If any of the characters (displayed as ASCII characters) are unprintable (control or meta characters) they will be replaced by periods.		
Insert a byte in decimal	`C-M-d`	(**hexl-insert-decimal-char** ARG)	Insert a character given by its decimal code ARG times at point.
Insert a byte in octal	`C-M-o`	(**hexl-insert-octal-char** ARG)	Insert a character given by its octal code ARG times at point.
Insert a byte in hex	`C-M-x`	(**hexl-insert-hex-char** ARG)	Insert a character given by its hexadecimal code ARG times at point.
Goto 512-byte page start	`C-M-a`	(**hexl-beginning-of-512b-page**)	Go to beginning of 512 byte boundary.
Goto to 512-byte page end	`C-M-e`	(**hexl-end-of-512b-page**)	Go to end of 512 byte boundary.
Goto 1K end	`C-x ]`	(**hexl-end-of-1k-page**)	Go to end of 1KB boundary.
Goto 1K beginning	`C-x [`	(**hexl-beginning-of-1k-page**)	Go to beginning of 1KB boundary.
Goto address entered in hexadecimal	`M-g`	(**hexl-goto-hex-address** HEX-ADDRESS)	Go to Hexl mode address (hex string) HEX-ADDRESS. • Signal error if HEX-ADDRESS is out of range.
Goto to address entered in decimal	`M-j`	(**hexl-goto-address** ADDRESS)	Go to hexl-mode (decimal) address ADDRESS. • Signal error if ADDRESS is out of range.
Exit hexl mode	`C-c C-c`	(**hexl-mode-exit** &optional ARG)	Exit Hexl mode, returning to previous mode. • With arg, don't unhexlify buffer.

Operation	Keystroke	Function	Note
Hexadecimal Editing with nhexl	The **nhexl-mode** external package used to display and manipulate the content of the current buffer in hexadecimal and manipulate hex dump files. PEL downloads installs and activates this package when the **pel-use-nhexl** user option is set to **t**. • Use the `<f11> b <f2>` key sequence to open the PEL buffer customization buffer to access this user option. Once the hexadecimal mode is on, turn it off by executing the **nhexl-mode** command again. Good **nhexl-mode** features: • The **nhexl-mode** keeps the undo history when you toggle the nhexl mode. Something that the helx mode does not do. • You can use all of the normal navigation commands. You don't need to use specialized commands. PEL **home** and **end** commands work.		
Toggle buffer between normal and hex display	`<f11> b x`	(**nhexl-mode** &optional ARG)	Toggle minor mode to edit files via hex-dump format. Requires the **nhexl-mode** package activated when **pel-use-nhexl** user option is **t**.
Activate Hex nibble editing mode	`<f11> b X`	(**nhexl-nibble-edit-mode** &optional ARG)	Minor mode to edit the hex nibbles in 'nhexl-mode'. Note: only works after nhexl-mode has been activated once. Requires the **nhexl-mode** package activated when **pel-use-nhexl** user option is **t**.
strace-mode **Colorize strace log output**	**pel-use-strace** : activates **strace-mode** external package used to display Linux **strace** output, required for this. • Type `<f1> o pel-use-strace` to open the customization buffer where you can activate it permanently.		
Turn on strace-mode	`<f11> b M-t`	(**strace-mode**)	Activate the strace-mode to colonize strace log output.